



NHẬP MÔN CƠ SỞ DỮ LIỆU

Nguyễn Hồng Hạnh

BM Công nghệ Phần mềm – Đại học Xây Dựng



Chương 4

MÔ HÌNH DỮ LIỆU QUAN HỆ

Nhập môn cơ sở dữ liệu



1. Khái niệm cơ bản của mô hình quan hệ

2. Chuyển lược đồ ER sang mô hình quan hệ

3. Đại số quan hệ

4. Ngôn ngữ SQL

1. Khái niệm cơ bản của mô hình quan hệ

1.1 Quan hệ

- Các thông tin lưu trữ trong CSDL được tổ chức thành bảng (table) 2 chiều gọi là quan hệ

1 cột là 1 thuộc tính của nhân viên

TENNV	HONV	NS	DIACHI	GT	LUONG	PHG
Tung	Nguyen	12/08/1955	638 NVC Q5	Nam	40000	5
Hang	Bui	07/19/1968	332 NTH Q1	Nu	25000	4
Nhu	Le	06/20/1951	291 HVH QPN	Nu	43000	4
Hung	Nguyen	09/15/1962	Ba Ria VT	Nam	38000	5

1 dòng là 1 nhân viên

Tên quan hệ là NHANVIEN



1. Khái niệm cơ bản của mô hình quan hệ

1.1 Quan hệ

- Quan hệ gồm
 - **Tên**
 - **Tập hợp các cột**
 - Cố định
 - Được đặt tên
 - Có kiểu dữ liệu
 - **Tập hợp các dòng**
 - Thay đổi theo thời gian
- Một dòng ~ Một thực thể
- Quan hệ ~ Tập thực thể

1. Khái niệm cơ bản của mô hình quan hệ

1.2 Miền giá trị

- Là tập các giá trị nguyên tố gắn liền với một thuộc tính
- Kiểu dữ liệu cơ sở
 - Chuỗi ký tự (string)
 - Số (integer)
- Các kiểu dữ liệu phức tạp
 - Tập hợp (set)
 - Danh sách (list)
 - Mảng (array)
 - Bản ghi (record)
- Ví dụ
 - TENV: string
 - LUONG: integer

} Không được chấp nhận

1. Khái niệm cơ bản của mô hình quan hệ

1.3 Định nghĩa hình thức

- Lược đồ quan hệ
 - Cho $A_1, A_2, \dots, A_n$ là các thuộc tính
 - Có các miền giá trị $D_1, D_2, \dots, D_n$ tương ứng
 - Ký hiệu $R(A_1:D_1, A_2:D_2, \dots, A_n:D_n)$ là một lược đồ quan hệ
 - **Bậc** của lược đồ quan hệ là số lượng thuộc tính trong lược đồ
- **Ví dụ:** NHANVIEN(MANV:integer, TENNV:string, HONV:string, NGSINH:date, DCHI:string, GT:string, LUONG:integer, DONVI:integer)
 - NHANVIEN là một lược đồ bậc 8 mô tả đối tượng nhân viên
 - MANV là một thuộc tính có miền giá trị là số nguyên
 - TENNV là một thuộc tính có miền giá trị là chuỗi ký tự

1. Khái niệm cơ bản của mô hình quan hệ

1.3 Định nghĩa hình thức

○ Quan hệ

- Một quan hệ r của lược đồ quan hệ $R(A_1, A_2, \dots, A_n)$, ký hiệu $r(R)$, là một tập các bộ $r = \{t_1, t_2, \dots, t_k\}$
- Trong đó mỗi t_i là 1 danh sách có thứ tự của n giá trị $t_i = \langle v_1, v_2, \dots, v_n \rangle$
- Mỗi v_j là một phần tử của miền giá trị $DOM(A_j)$ hoặc giá trị rỗng

	TENNV	HONV	NGSINH	DCHI	PHAI	LUONG	PHG
t_1	Tung	Nguyen	12/08/1955	638 NVC Q5	Nam	40000	5
t_2	Hang	Bui	07/19/1968	332 NTH Q1	Nu	25000	4
t_3	Nhu	Le	06/20/1951	291 HVH QPN	Nu	43000	4
t_4	Hung	Nguyen	09/15/1962	null	Nam	38000	5

v_i

1. Khái niệm cơ bản của mô hình quan hệ

1.4 Tóm tắt các ký hiệu

- Lược đồ quan hệ R bậc n
 - $R(A_1, A_2, \dots, A_n)$
- Tập thuộc tính của R
 - R^+
- Quan hệ (thể hiện quan hệ)
 - R, S, P, Q
- Bộ
 - t, u, v
- Miền giá trị của thuộc tính A
 - $DOM(A)$ hay $MGT(A)$
- Giá trị tại thuộc tính A của bộ thứ t
 - $t.A$ hay $t[A]$

1. Khái niệm cơ bản của mô hình quan hệ

1.5 Khái niệm về khóa

- Định nghĩa
 - Gọi K là một tập con khác rỗng các thuộc tính của R
 - K là khóa nếu thỏa đồng thời 2 điều kiện:
 - $\forall r, \forall t_1, t_2 \in r, t_1 \neq t_2 \Rightarrow t_1[K] \neq t_2[K]$
 - $\forall K' \subset K, K' \neq K, K'$ không phải là khóa của R
- Nhận xét
 - Giá trị của khóa dùng để nhận biết một bộ trong quan hệ
 - Khóa là một đặc trưng của lược đồ quan hệ, không phụ thuộc vào thể hiện quan hệ
 - Khóa được xây dựng dựa vào ý nghĩa của một số thuộc tính trong quan hệ
 - Lược đồ quan hệ có thể có nhiều khóa

1. Khái niệm cơ bản của mô hình quan hệ

1.5 Khái niệm về khóa

- Khóa chính
 - Xét quan hệ
NHANVIEN(MANV, TENNV, HONV, NS, DCHI, GT, LUONG, PHG)
 - Có 2 khóa
 - MANV
 - HONV, TENNV, NS
 - Khi cài đặt quan hệ thành bảng (table)
 - Chọn 1 khóa làm cơ sở để nhận biết các bộ
 - Khóa có ít thuộc tính hơn
 - Khóa được chọn gọi là khóa chính (PK - primary key)
 - Các thuộc tính khóa chính phải có giá trị khác null
 - Các thuộc tính khóa chính thường được gạch dưới
- NHANVIEN(MANV, TENNV, HONV, NS, DCHI, GT, LUONG, PHG)

1. Khái niệm cơ bản của mô hình quan hệ

1.5 Khái niệm về khóa

- Tham chiếu

- Một bộ trong quan hệ R, tại thuộc tính A nếu nhận một giá trị từ một thuộc tính B của quan hệ S, ta gọi R tham chiếu S
- Bộ được tham chiếu phải tồn tại trước

S

TENPHG	MAPHG
Nghien cuu	5
Dieu hanh	4
Quan ly	1

R

TENNV	HONV	NS	DCHI	GT	LUONG	PHG
Tung	Nguyen	12/08/1955	638 NVC Q5	Nam	40000	5
Hang	Bui	07/19/1968	332 NTH Q1	Nu	25000	4
Nhu	Le	06/20/1951	291 HVH QPN	Nu	43000	4
Hung	Nguyen	09/15/1962	Ba Ria VT	Nam	38000	5

1. Khái niệm cơ bản của mô hình quan hệ

1.5 Khái niệm về khóa

○ Khóa ngoại

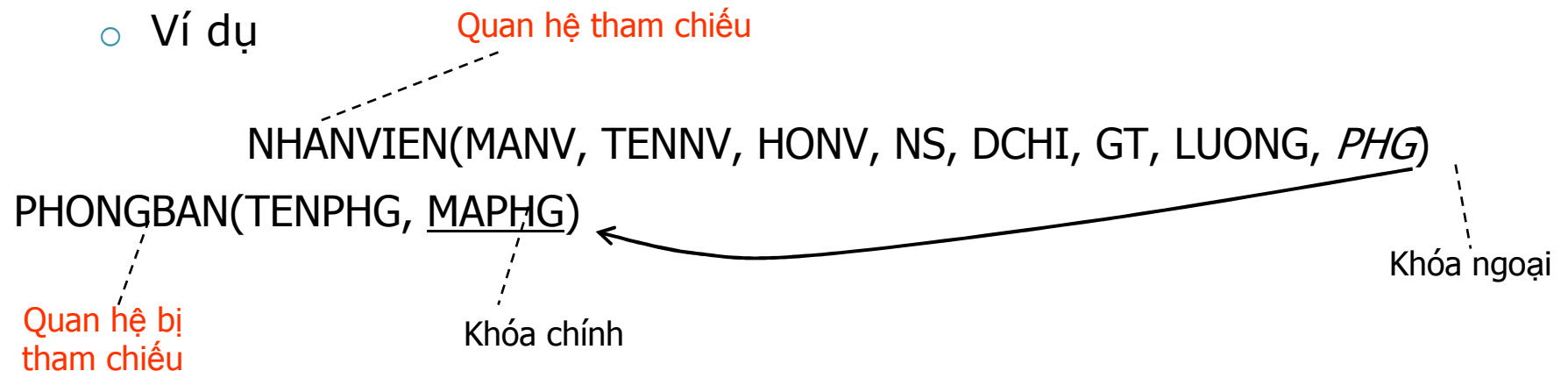
○ Xét 2 lược đồ R và S

- Gọi FK là tập thuộc tính khác rỗng của R
- FK là khóa ngoại (Foreign Key) của R khi
- Các thuộc tính trong FK phải có cùng miền giá trị với các thuộc tính khóa chính của S

○ Giá trị tại FK của một bộ $t_1 \in R$

- Hoặc bằng giá trị tại khóa chính của một bộ $t_2 \in S$
- Hoặc bằng giá trị rỗng

○ Ví dụ



1. Khái niệm cơ bản của mô hình quan hệ

1.6 Các đặc trưng của quan hệ

- Thứ tự các bộ trong quan hệ là không quan trọng

HONV	TENNV	NGSINH	DCHI	PHAI	LUONG	PHG
Bui	Hang	07/19/1968	332 NTH Q1	Nu	25000	4
Le	Nhu	06/20/1951	291 HVH QPN	Nu	43000	4

- Thứ tự giữa các giá trị trong một bộ là quan trọng

Bộ <Nguyen, Tung, 12/08/1955, 638 NVC Q5, **Nam**, **40000**, 5>

khác

Bộ <Nguyen, Tung, 12/08/1955, 638 NVC Q5, **40000**, **Nam**, 5>

- Không có bộ nào trùng nhau

Nhập môn cơ sở dữ liệu

1. Khái niệm cơ bản của mô hình quan hệ



2. Chuyển lược đồ ER sang mô hình quan hệ

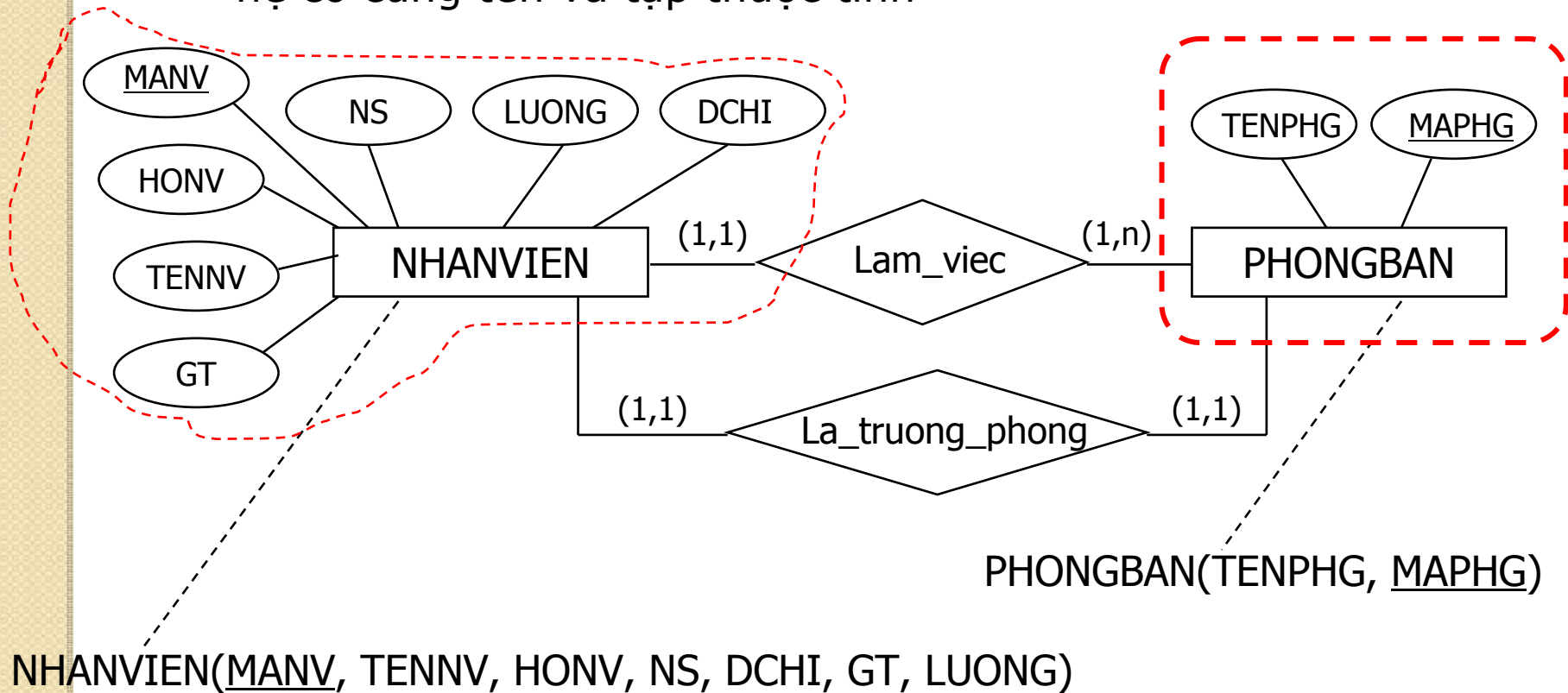
3. Ngôn ngữ đại số quan hệ

4. Ngôn ngữ SQL

2. Chuyển lược đồ ER sang mô hình quan hệ

Các quy tắc chuyển đổi

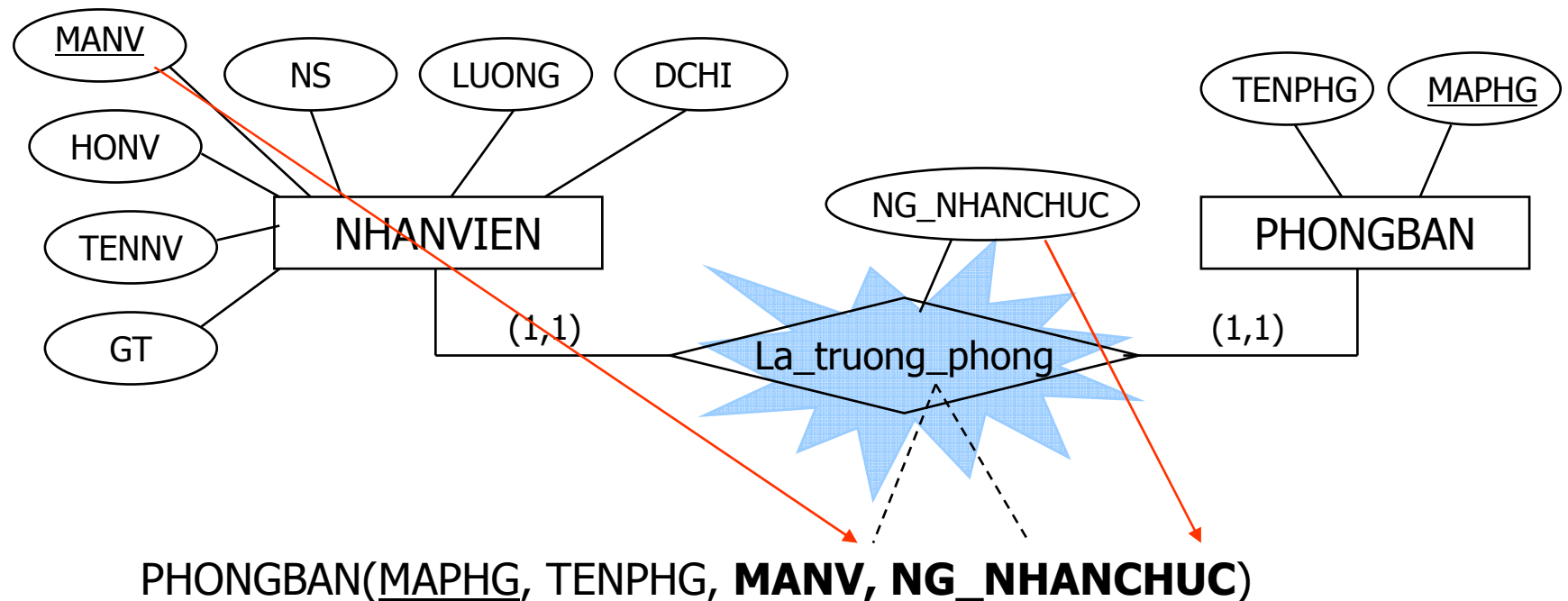
- (1) Tập thực thể
 - Các tập thực thể (trừ tập thực thể yếu) chuyển thành các quan hệ có cùng tên và tập thuộc tính



2. Chuyển lược đồ ER sang mô hình quan hệ

Các quy tắc chuyển đổi

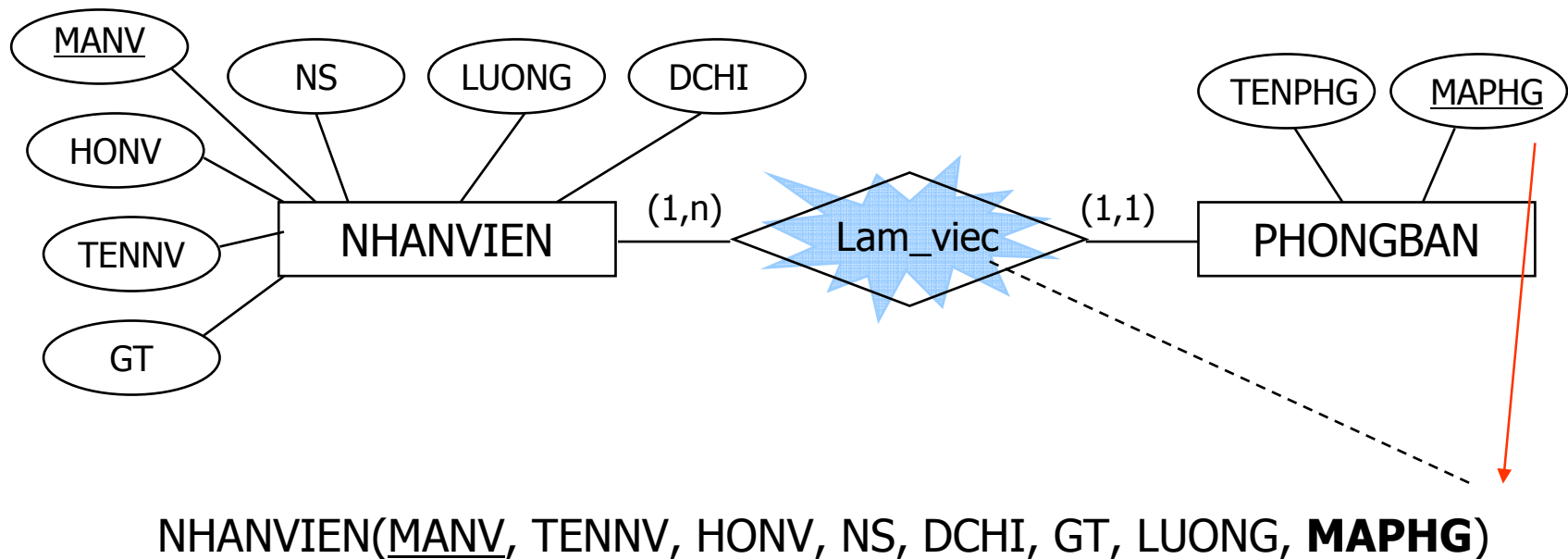
- (2) Mỗi quan hệ
 - (2a) Một-Một
 - Hoặc thêm vào quan hệ này thuộc tính khóa của quan hệ kia
 - Hoặc thêm thuộc tính khóa vào cả 2 quan hệ



2. Chuyển lược đồ ER sang mô hình quan hệ

Các quy tắc chuyển đổi

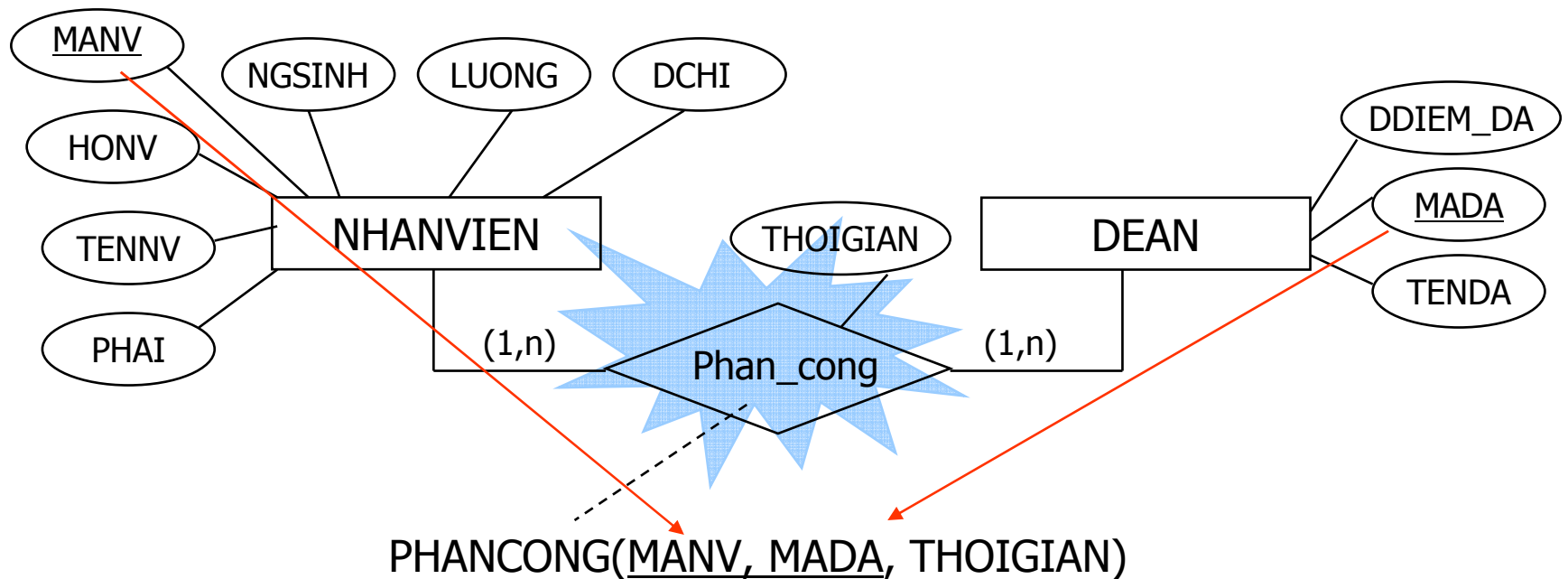
- (2) Mỗi quan hệ
 - (2b) Một-Nhiều
 - Thêm vào quan-hệ-nhiều thuộc tính khóa của quan-hệ-một
(Lấy khóa chính của quan hệ bên 1 đưa vào làm khóa ngoài của quan hệ bên n)



2. Chuyển lược đồ ER sang mô hình quan hệ

Các quy tắc chuyển đổi

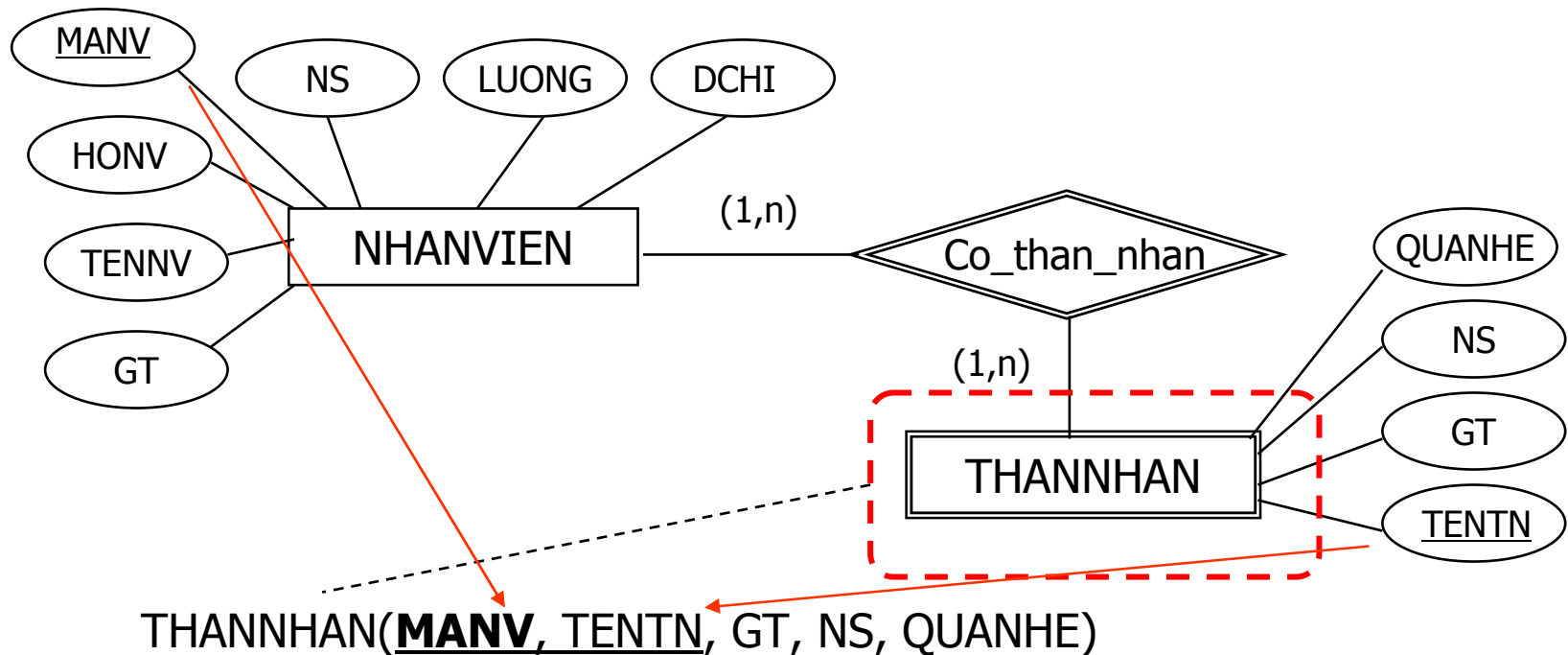
- (2) Mỗi quan hệ
 - (2c) Nhiều-Nhiều
 - Tạo một quan hệ mới có
 - Tên quan hệ là tên của mối quan hệ
 - Thuộc tính là những thuộc tính khóa của các tập thực thể liên quan



2. Chuyển lược đồ ER sang mô hình quan hệ

Các quy tắc chuyển đổi

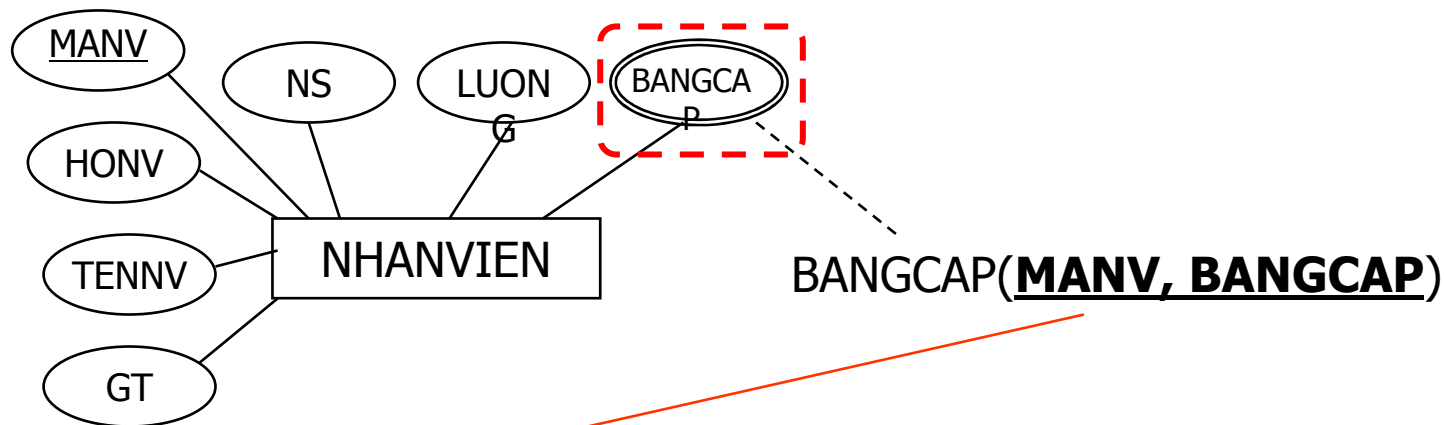
- (3) Thực thể yếu
 - Chuyển thành một quan hệ
 - Có cùng tên với thực thể yếu
 - Thêm vào thuộc tính khóa của quan hệ liên quan



2. Chuyển lược đồ ER sang mô hình quan hệ

Các quy tắc chuyển đổi

- (4) Thuộc tính đa trị
 - Chuyển thành một quan hệ
 - Có cùng tên với thuộc tính đa trị
 - Thuộc tính khóa của quan hệ này được xác định bởi thuộc tính đa trị và khóa của tập thực thể tương ứng.

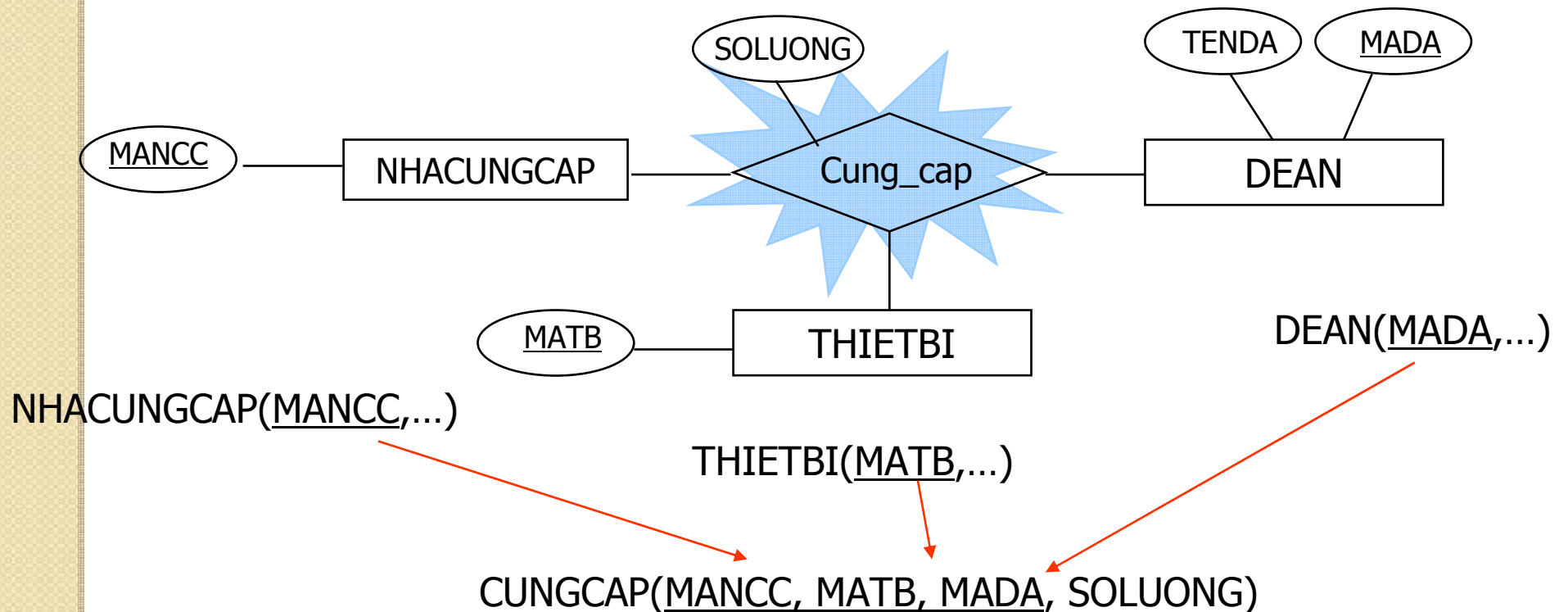


NHANVIEN(MANV, TENNV, HONV, NS, DCHI, GT, LUONG, DCHI)

2. Chuyển lược đồ ER sang mô hình quan hệ

Các quy tắc chuyển đổi

- (5) Liên kết đa ngôi ($n > 2$)
 - Chuyển thành một quan hệ
 - Có cùng tên với tên mỗi liên kết đa ngôi
 - Khóa chính là tổ hợp các khóa của tập các thực thể tham gia liên kết



Nhập môn cơ sở dữ liệu

1. Khái niệm cơ bản của mô hình quan hệ

2. Chuyển lược đồ ER sang mô hình quan hệ



3. Ngôn ngữ đại số quan hệ

4. Ngôn ngữ SQL

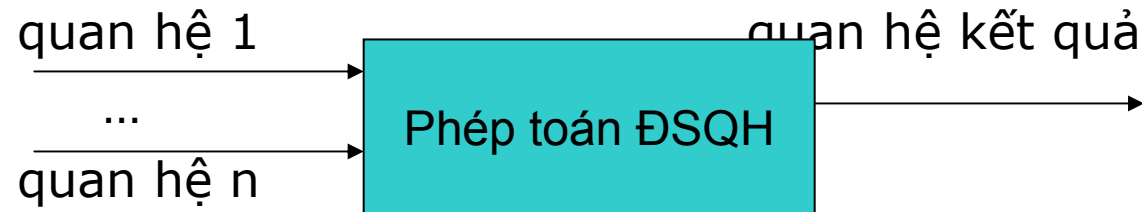
3. Ngôn ngữ đại số quan hệ

3.1 Tổng quan về ngôn ngữ đại số quan hệ

- Ngôn ngữ ĐSQH gồm các phép toán tương ứng với các thao tác trên các quan hệ (bảng).
- Mỗi phép toán đại số quan hệ

Đầu vào: Một hay nhiều quan hệ

Đầu ra : Một quan hệ



(Kết quả thực hiện một phép toán ĐSQH là một quan hệ)

- ***Biểu thức đại số quan hệ*** : chính là một chuỗi các phép toán
- Kết quả thực hiện một biểu thức đại số quan hệ cũng là một quan hệ.
- Được cài đặt trong phần lớn các hệ CSDL hiện nay.

3. Ngôn ngữ đại số quan hệ

3.2 Phân loại các phép toán

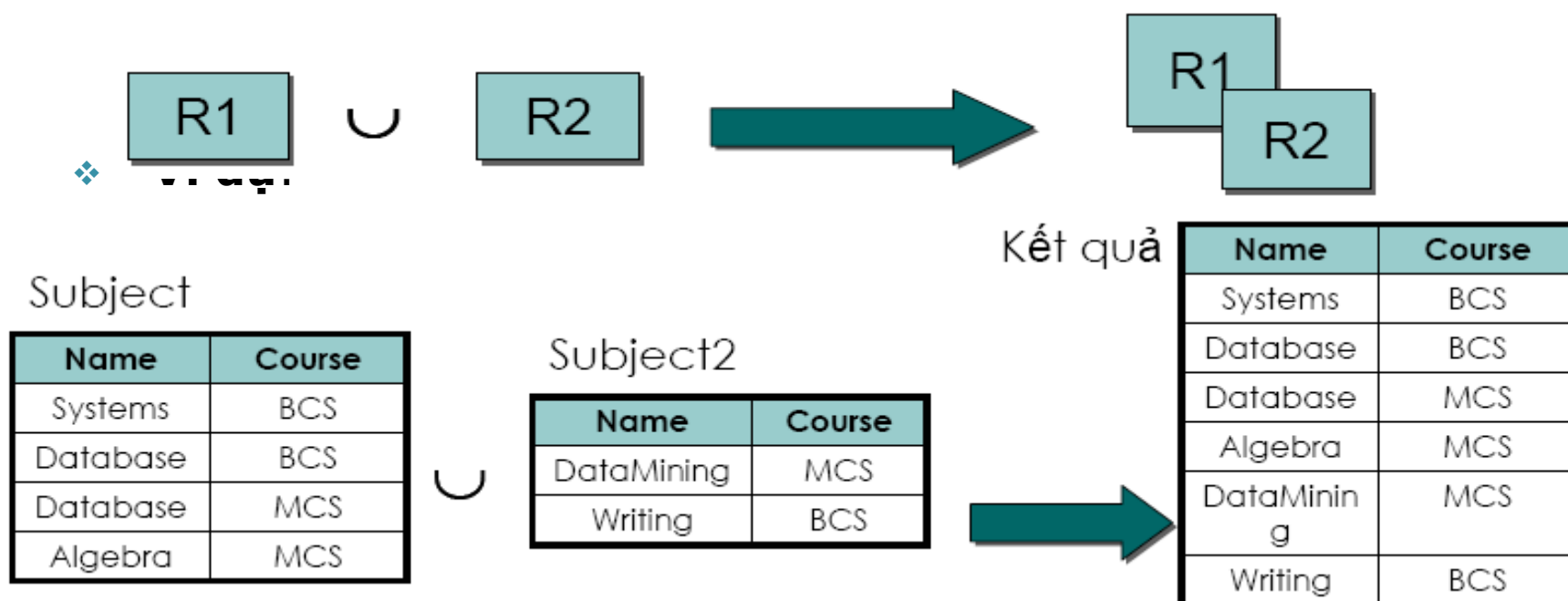
- Phép toán quan hệ:
 - Phép chiếu (*projection*)
 - Phép chọn (*selection*)
 - Phép kết nối (*join*)
 - Phép chia (*division*)
- Phép toán tập hợp
 - Phép hợp(*union*)
 - Phép giao(*intersection*)
 - Phép trừ (*difference*)
 - Phép tích Đề-các (*cartesian product*)

3. Ngôn ngữ đại số quan hệ

3.3 Phép toán tập hợp

3.3.1 Phép hợp (union)

- ❖ **Định nghĩa:** Gồm các bộ thuộc ít nhất một trong hai quan hệ đầu vào. Hai quan hệ đầu vào phải là hai quan hệ khả hợp
2 quan hệ khả hợp được xác định trên cùng miền giá trị
- ❖ **Cú pháp :** $R_1 \cup R_2$



3. Ngôn ngữ đại số quan hệ

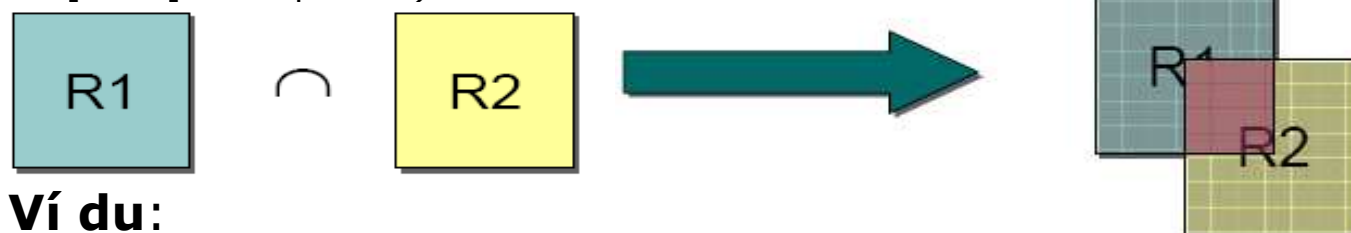
3.3 Phép toán tập hợp

3.3.2 Phép giao

- ❖ **Định nghĩa:** Gồm các bộ thuộc cả hai quan hệ đầu vào. Hai quan hệ đầu vào phải là hai quan hệ khả hợp

2 quan hệ khả hợp được xác định trên cùng miền giá trị

- ❖ **Cú pháp :** $R_1 \cap R_2$



- ❖ **Ví dụ:**

Subject		Subject2		Kết quả	
Name	Course	Name	Course	Name	Course
Systems	BCS	DataMining	MCS	Systems	BCS
Database	BCS	Database	MCS	Database	MCS
Database	MCS	Systems	BCS		
Algebra	MCS	Writing	BCS		

3. Ngôn ngữ đại số quan hệ

3.3 Phép toán tập hợp

3.3.3 Phép trừ

- ❖ **Định nghĩa:** Gồm các bộ thuộc quan hệ thứ nhất nhưng không thuộc quan hệ thứ hai. Hai quan hệ đầu vào phải là hai quan hệ khả hợp

2 quan hệ khả hợp được xác định trên cùng miền giá trị

- ❖ **Cú pháp :** $R_1 \setminus R_2$



- ❖ **Ví dụ:**

Subject

Name	Course
Systems	BCS
Database	BCS
Database	MCS
Algebra	MCS

Subject2

Name	Course
DataMining	MCS
Database	MCS
Systems	BCS
Writing	BCS

\



Kết quả

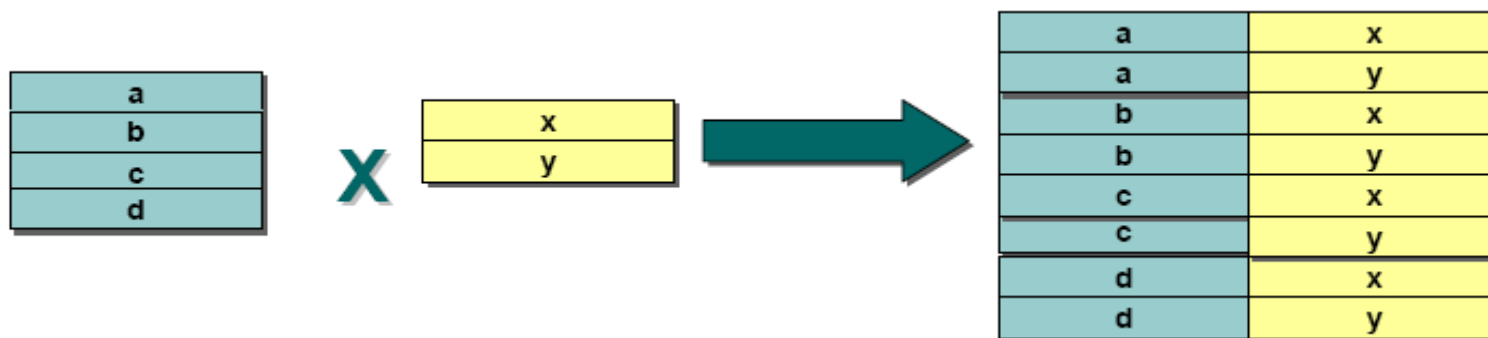
Name	Course
Database	BCS
Algebra	MCS

3. Ngôn ngữ đại số quan hệ

3.3 Phép toán tập hợp

3.3.4 Phép tích Descartes

- ❖ **Định nghĩa:** Là kết nối giữa từng bộ của quan hệ thứ nhất với mỗi bộ của quan hệ thứ hai
- ❖ **Cú pháp :** $R_1 \times R_2$



3. Ngôn ngữ đại số quan hệ

3.3 Phép toán tập hợp

3.3.4 Phép tích Descartes

❖ Ví dụ:

Student

Id	Name	Suburb
1108	Robert	Kew
3936	Glen	Bundoora
8507	Norman	Bundoora
8452	Mary	Balwyn

X

Sport

SportID	Sport
05	Swimming
09	Dancing

Student_Sport



Id	Name	Suburb	SportID	Sport
1108	Robert	Kew	05	Swimming
3936	Glen	Bundoora	05	Swimming
8507	Norman	Bundoora	05	Swimming
8452	Mary	Balwyn	05	Swimming
1108	Robert	Kew	09	Dancing
3936	Glen	Bundoora	09	Dancing
8507	Norman	Bundoora	09	Dancing
8452	Mary	Balwyn	09	Dancing

3. Ngôn ngữ đại số quan hệ

3.4 Phép toán quan hệ

3.4.1 Phép chiếu

- **Định nghĩa:** Lựa chọn một số thuộc tính từ một quan hệ
- **Cú pháp :** $\Pi_{A_1, A_2, \dots, A_k}(R)$
- Kết quả trả về là một quan hệ
 - Có k thuộc tính ($k \leq n$ với n là số thuộc tính của R)



❖ Ví dụ: đưa ra danh sách tên của tất cả các sinh viên

$$\Pi_{name}(Student)$$

Student					Kết quả
Id	Name	Suburb			Name
1108	Robert	Kew			Robert
3936	Glen	Bundoora			Glen
8507	Norman	Bundoora			Norman
8452	Mary	Balwyn			Mary

3. Ngôn ngữ đại số quan hệ

3.4 Phép toán quan hệ

3.4.2 Phép chọn

- **Định nghĩa:** Lựa chọn các bộ trong một quan hệ thỏa mãn điều kiện cho trước
- **Cú pháp :** $\sigma_{\langle \text{condition} \rangle} (R)$



- **Ví dụ :** đưa ra danh sách những sinh viên sống ở Bundoora

$$\sigma_{\text{suburb}=\text{"Bundoora"}} (\text{Student})$$

Student

Id	Name	Suburb
1108	Robert	Kew
3936	Glen	Bundoora
8507	Norman	Bundoora
8452	Mary	Balwyn

Kết quả

Id	Name	Suburb
3936	Glen	Bundoora
8507	Norman	Bundoora

1. Ngôn ngữ đại số quan hệ

Kết hợp phép chọn và chiếu

Đưa ra tên các sinh viên sống ở Bundoora

$$\Pi_{name}(\sigma_{suburb="Bundoora"}Student)$$

Student

Id	Name	Suburb
1108	Robert	Kew
3936	Glen	Bundoora
8507	Norman	Bundoora
8452	Mary	Balwyn



Kết quả

Name
Glen
Norman

3. Ngôn ngữ đại số quan hệ

3.4 Phép toán quan hệ

3.4.3 Phép kết nối

- Định nghĩa:** ghép các bộ từ 2 quan hệ thỏa mãn điều kiện kết nối.

$$R_1 \bowtie_{\langle \text{join_condition} \rangle} R_2$$

- Cú pháp:**



- Ví dụ :** Đưa ra danh sách các sinh viên và khóa học mà họ đang theo học .

$$\text{Student} \bowtie_{\text{Id}=\text{SID}} \text{Enrol}$$

Student

Id	Name	Suburb
1108	Robert	Kew
3936	Glen	Bundoora
8507	Norman	Bundoora
8452	Mary	Balwyn

Id=SID

Enrol

SID	Course
3936	101
1108	113
8507	101

Kết quả

Id	Name	Suburb	Course
1108	Robert	Kew	113
3936	Glen	Bundoora	101
8507	Norman	Bundoora	101

3. Ngôn ngữ đại số quan hệ

Kết hợp phép chọn, chiếu và kết nối

Đưa ra tên các sinh viên sống ở Bundoora và mã khóa học mà sinh viên đó đã đăng ký.

$$\Pi_{name, Course} (\sigma_{suburb="Bundoora"} (Student \bowtie_{Id=SID} Enrol))$$

Student

Id	Name	Suburb
1108	Robert	Kew
3936	Glen	Bundoora
8507	Norman	Bundoora
8452	Mary	Balwyn

Enrol

SID	Course
3936	101
1108	113
8507	101



Kết quả

Name	Course
Glen	101
Norman	101

3. Ngôn ngữ đại số quan hệ

3.4 Phép toán quan hệ

3.4.4 Phép kết nối tự nhiên

- ❖ **Định nghĩa:** Là phép kết nối với điều kiện bằng trên các thuộc tính trùng tên. Trong một số tài liệu, kí hiệu * được dùng để biểu thị phép kết nối tự nhiên.
- ❖ **Ví dụ**

Takes

SID	SNO
1108	21
1108	23
8507	23
8507	29

Enrol

SID	Course
3936	101
1108	113
8507	101



SID	SNO	Course
1108	21	113
1108	23	113
8507	23	101
8507	29	101

3. Ngôn ngữ đại số quan hệ

3.4 Phép toán quan hệ

3.4.4 Phép kết nối

a. Phép kết nối ngoài trái



b. Phép kết nối ngoài phải



3. Ngôn ngữ đại số quan hệ

3.4 Phép toán quan hệ

3.4.4 Phép kết nối

❖ Ví dụ về phép kết nối ngoài:

Đưa ra danh sách mã số các sinh viên và mã khóa học mà sinh viên đó đăng ký nếu có.

Student

ID	Name	Suburb
1108	Robert	Kew
3936	Glen	Bundoora
8507	Norman	Bundoora
8452	Mary	Balwyn

Enrol

SID	Course
3936	101
1108	113
8507	101



Kết quả



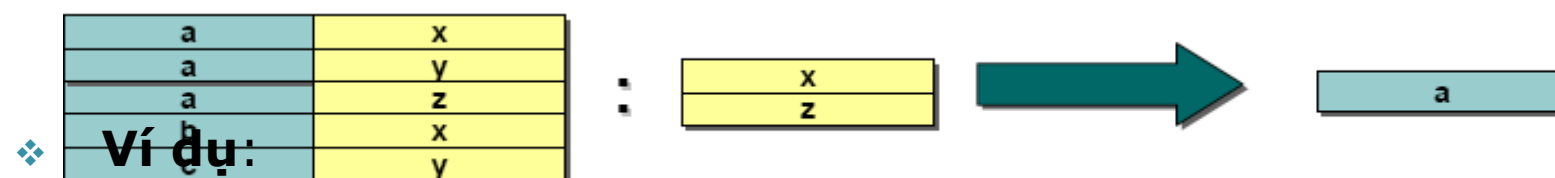
ID	Name	Suburb	Course
1108	Robert	Kew	113
3936	Glen	Bundoora	101
8507	Norman	Bundoora	101
8452	Mary	Balwyn	null

3. Ngôn ngữ đại số quan hệ

3.4 Phép toán quan hệ

3.4.5 Phép chia

- ❖ **Định nghĩa:** Cho R_1 và R_2 lần lượt là các quan hệ n và m ngôi. Kết quả của phép chia R_1 cho R_2 là một quan hệ $(n-m)$ ngôi.
- ❖ **Cú pháp :** $R_1 : R_2$



Subject

Name	Course
Systems	BCS
Database	BCS
Database	MCS
Algebra	MCS

:

Course

Course
BCS
MCS



Kết quả

Name
Database

Nhập môn cơ sở dữ liệu

1. Khái niệm cơ bản của mô hình quan hệ
2. Chuyển lược đồ ER sang mô hình quan hệ
3. Ngôn ngữ đại số quan hệ



4. Ngôn ngữ SQL

4. Ngôn ngữ SQL

4.1 Tổng quan

- ❖ Ngôn ngữ Đại số quan hệ
 - ❖ Cách thức truy vấn dữ liệu
 - ❖ Khó khăn cho người sử dụng
- ❖ SQL (Structured Query Language)
 - ❖ Ngôn ngữ cấp cao
 - ❖ Người sử dụng chỉ cần đưa ra nội dung cần truy vấn
 - ❖ Được phát triển bởi IBM (1970s)
 - ❖ Được gọi là SEQUEL
 - ❖ Được ANSI công nhận và phát triển thành chuẩn
 - ❖ SQL-86
 - ❖ SQL-92
 - ❖ SQL-99

4. Ngôn ngữ SQL

4.1 Tổng quan

- ❖ SQL gồm
 - ❖ Định nghĩa dữ liệu (DDL)
 - ❖ Thao tác dữ liệu (DML)
 - ❖ Định nghĩa khung nhìn
 - ❖ Ràng buộc toàn vẹn
 - ❖ Phân quyền và bảo mật
 - ❖ Điều khiển giao tác
- ❖ SQL sử dụng thuật ngữ
 - ❖ Bảng ~ quan hệ
 - ❖ Cột ~ thuộc tính
 - ❖ Dòng ~ bộ

4. Ngôn ngữ SQL

4.2 Định nghĩa dữ liệu

- ❖ Là ngôn ngữ mô tả
 - ❖ Lược đồ cho mỗi quan hệ
 - ❖ Miền giá trị tương ứng của từng thuộc tính
 - ❖ Ràng buộc toàn vẹn
 - ❖ Chỉ mục trên mỗi quan hệ
- ❖ Gồm
 - ❖ CREATE TABLE (tạo bảng)
 - ❖ DROP TABLE (xóa bảng)
 - ❖ ALTER TABLE (sửa bảng)
 - ❖ CREATE DOMAIN (tạo miền giá trị)
 - ❖ CREATE DATABASE
 - ❖ ...

4. Ngôn ngữ SQL

4.2 Kiểu dữ liệu

- ❖ Số (numeric)
 - ❖ INTEGER
 - ❖ SMALLINT
 - ❖ NUMERIC, NUMERIC(p), NUMERIC(p,s)
 - ❖ DECIMAL, DECIMAL(p), DECIMAL(p,s)
 - ❖ REAL
 - ❖ DOUBLE PRECISION
 - ❖ FLOAT, FLOAT(p)

4. Ngôn ngữ SQL

4.2 Kiểu dữ liệu

- ❖ Chuỗi ký tự (character string)
 - ❖ CHARACTER, CHARACTER(n)
 - ❖ CHARACTER VARYING(x)
- ❖ Chuỗi bit (bit string)
 - ❖ BIT, BIT(x)
 - ❖ BIT VARYING(x)
- ❖ Ngày giờ (datetime)
 - ❖ DATE gồm ngày, tháng và năm
 - ❖ TIME gồm giờ, phút và giây
 - ❖ TIMESTAMP gồm ngày và giờ

4. Ngôn ngữ SQL

Microsoft Access Data Types

Data type	Description	Storage
Text	Use for text or combinations of text and numbers. 255 characters maximum	
Memo	Memo is used for larger amounts of text. Stores up to 65,536 characters. Note: You cannot sort a memo field. However, they are searchable	
Byte	Allows whole numbers from 0 to 255	1 byte
Integer	Allows whole numbers between -32,768 and 32,767	2 bytes
Long	Allows whole numbers between -2,147,483,648 and 2,147,483,647	4 bytes
Single	Single precision floating-point. Will handle most decimals	4 bytes
Double	Double precision floating-point. Will handle most decimals	8 bytes
Currency	Use for currency. Holds up to 15 digits of whole dollars, plus 4 decimal places. Tip: You can choose which country's currency to use	8 bytes
AutoNumber	AutoNumber fields automatically give each record its own number, usually starting at 1	4 bytes
Date/Time	Use for dates and times	8 bytes
Yes/No	A logical field can be displayed as Yes/No, True/False, or On/Off. In code, use the constants True and False (equivalent to -1 and 0). Note: Null values are not allowed in Yes/No fields	1 bit
Ole Object	Can store pictures, audio, video, or other BLOBs (Binary Large Objects)	up to 1GB
Hyperlink	Contain links to other files, including web pages	
Lookup Wizard	Let you type a list of options, which can then be chosen from a drop-down list	4 bytes

4. Ngôn ngữ SQL

MySQL Data Types

Text types:

Data type	Description
CHAR(size)	Holds a fixed length string (can contain letters, numbers, and special characters). The fixed size is specified in parenthesis. Can store up to 255 characters
VARCHAR(size)	Holds a variable length string (can contain letters, numbers, and special characters). The maximum size is specified in parenthesis. Can store up to 255 characters. Note: If you put a greater value than 255 it will be converted to a TEXT type
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT	Holds a string with a maximum length of 65,535 characters
BLOB	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LOBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
ENUM(x,y,z,etc.)	Let you enter a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. Note: The values are sorted in the order you enter them. You enter the possible values in this format: ENUM('X','Y','Z')
SET	Similar to ENUM except that SET may contain up to 64 list items and can store more than one choice

4. Ngôn ngữ SQL

MySQL Data Types

Number types:

Data type	Description
TINYINT(size)	-128 to 127 normal. 0 to 255 UNSIGNED*. The maximum number of digits may be specified in parenthesis
SMALLINT(size)	-32768 to 32767 normal. 0 to 65535 UNSIGNED*. The maximum number of digits may be specified in parenthesis
MEDIUMINT(size)	-8388608 to 8388607 normal. 0 to 16777215 UNSIGNED*. The maximum number of digits may be specified in parenthesis
INT(size)	-2147483648 to 2147483647 normal. 0 to 4294967295 UNSIGNED*. The maximum number of digits may be specified in parenthesis
BIGINT(size)	-9223372036854775808 to 9223372036854775807 normal. 0 to 18446744073709551615 UNSIGNED*. The maximum number of digits may be specified in parenthesis
FLOAT(size,d)	A small number with a floating decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter
DOUBLE(size,d)	A large number with a floating decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter
DECIMAL(size,d)	A DOUBLE stored as a string , allowing for a fixed decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter

4. Ngôn ngữ SQL

MySQL Data Types

Date types:

Data type	Description
DATE()	A date. Format: YYYY-MM-DD Note: The supported range is from '1000-01-01' to '9999-12-31'
DATETIME()	*A date and time combination. Format: YYYY-MM-DD HH:MM:SS Note: The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'
TIMESTAMP()	*A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD HH:MM:SS Note: The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC
TIME()	A time. Format: HH:MM:SS Note: The supported range is from '-838:59:59' to '838:59:59'
YEAR()	A year in two-digit or four-digit format. Note: Values allowed in four-digit format: 1901 to 2155. Values allowed in two-digit format: 70 to 69, representing years from 1970 to 2069

4. Ngôn ngữ SQL

SQL Server Data Types

Character strings:

Data type	Description	Storage
char(n)	Fixed-length character string. Maximum 8,000 characters	n
varchar(n)	Variable-length character string. Maximum 8,000 characters	
varchar(max)	Variable-length character string. Maximum 1,073,741,824 characters	
text	Variable-length character string. Maximum 2GB of text data	

Unicode strings:

Data type	Description	Storage
nchar(n)	Fixed-length Unicode data. Maximum 4,000 characters	
nvarchar(n)	Variable-length Unicode data. Maximum 4,000 characters	
nvarchar(max)	Variable-length Unicode data. Maximum 536,870,912 characters	
ntext	Variable-length Unicode data. Maximum 2GB of text data	

Binary types:

Data type	Description	Storage
bit	Allows 0, 1, or NULL	
binary(n)	Fixed-length binary data. Maximum 8,000 bytes	
varbinary(n)	Variable-length binary data. Maximum 8,000 bytes	
varbinary(max)	Variable-length binary data. Maximum 2GB	
image	Variable-length binary data. Maximum 2GB	

4. Ngôn ngữ SQL

Number types:

Data type	Description	Storage
tinyint	Allows whole numbers from 0 to 255	1 byte
smallint	Allows whole numbers between -32,768 and 32,767	2 bytes
int	Allows whole numbers between -2,147,483,648 and 2,147,483,647	4 bytes
bigint	Allows whole numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807	8 bytes
decimal(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
numeric(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
smallmoney	Monetary data from -214,748.3648 to 214,748.3647	4 bytes
money	Monetary data from -922,337,203,685,477.5808 to 922,337,203,685,477.5807	8 bytes
float(n)	Floating precision number data from $-1.79E + 308$ to $1.79E + 308$. The n parameter indicates whether the field should hold 4 or 8 bytes. float(24) holds a 4-byte field and float(53) holds an 8-byte field. Default value of n is 53.	4 or 8 bytes
real	Floating precision number data from $-3.40E + 38$ to $3.40E + 38$	4 bytes

4. Ngôn ngữ SQL

Date types:

Data type	Description	Storage
datetime	From January 1, 1753 to December 31, 9999 with an accuracy of 3.33 milliseconds	8 bytes
datetime2	From January 1, 0001 to December 31, 9999 with an accuracy of 100 nanoseconds	6-8 bytes
smalldatetime	From January 1, 1900 to June 6, 2079 with an accuracy of 1 minute	4 bytes
date	Store a date only. From January 1, 0001 to December 31, 9999	3 bytes
time	Store a time only to an accuracy of 100 nanoseconds	3-5 bytes
datetimeoffset	The same as datetime2 with the addition of a time zone offset	8-10 bytes
timestamp	Stores a unique number that gets updated every time a row gets created or modified. The timestamp value is based upon an internal clock and does not correspond to real time. Each table may have only one timestamp variable	

Other data types:

Data type	Description
sql_variant	Stores up to 8,000 bytes of data of various data types, except text, ntext, and timestamp
uniqueidentifier	Stores a globally unique identifier (GUID)
xml	Stores XML formatted data. Maximum 2GB
cursor	Stores a reference to a cursor used for database operations
table	Stores a result-set for later processing

4. Ngôn ngữ SQL

4.3 *Lệnh tạo bảng*

- ❖ Để định nghĩa một bảng
 - ❖ Tên bảng
 - ❖ Các thuộc tính
 - ❖ Tên thuộc tính
 - ❖ Kiểu dữ liệu
 - ❖ Các ràng buộc toàn vẹn trên thuộc tính (RBTV)
- ❖ Cú pháp

```
CREATE TABLE <Tên_bảng> (  
    <Tên_cột> <Kiểu_dữ_liệu> [<RBTV>],  
    <Tên_cột> <Kiểu_dữ_liệu> [<RBTV>],  
    ...  
    [<RBTV>]  
)
```

4. Ngôn ngữ SQL

4.3 Lệnh tạo bảng

❖ Ví dụ

```
CREATE TABLE NHANVIEN (  
    MANV CHAR(9),  
    HONV VARCHAR(10),  
    TENDEM VARCHAR(20),  
    TENNV VARCHAR(10),  
    NS     DATETIME,  
    DCHI VARCHAR(50),  
    GT CHAR(3),  
    LUONG INT,  
    MA_NQL CHAR(9),  
    PHG INT  
)
```

4. Ngôn ngữ SQL

4.3 Lệnh tạo bảng

- ❖ Các ràng buộc toàn vẹn
 - ❖ NOT NULL
 - ❖ NULL
 - ❖ UNIQUE
 - ❖ DEFAULT
 - ❖ PRIMARY KEY
 - ❖ FOREIGN KEY / REFERENCES
 - ❖ CHECK
- ❖ Đặt tên cho ràng buộc

CONSTRAINT <Ten_RBTv> <RBTv>

4. Ngôn ngữ SQL

4.3 *Lệnh tạo bảng*

❖ Các ràng buộc toàn vẹn

❖ NOT NULL

Ràng buộc NOT NULL buộc một cột không cho phép nhận giá trị rỗng. Nghĩa là không thể thực hiện thao tác thêm hay cập nhật một bản ghi mà chưa điền giá trị vào cột được thiết lập ràng buộc NOT NULL.

Ví dụ:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255)
)
```

4. Ngôn ngữ SQL

Các ràng buộc toàn vẹn

❖ UNIQUE

Ràng buộc UNIQUE xác định sự duy nhất của một bản ghi trong bảng.

Ví dụ:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL UNIQUE,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255)
)
```

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT uc_PersonID UNIQUE (P_Id,LastName)
)
```

4. Ngôn ngữ SQL

Các ràng buộc toàn vẹn

❖ DEFAULT

Ràng buộc DEFAULT thiết lập giá trị mặc định cho một cột trong bảng. Khi thêm một bản ghi mới, giá trị mặc định này sẽ được tự động điền vào cột tương ứng nếu như cột đó không được xác định giá trị. Ví dụ:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255) DEFAULT 'Sandnes'
)
```

Ràng buộc DEFAULT còn
dùng để thêm

hàm như GETDATE() để hỗ trợ việc sử dụng kèm các

```
CREATE TABLE Orders
(
  O_Id int NOT NULL,
  OrderNo int NOT NULL,
  P_Id int,
  OrderDate date DEFAULT GETDATE()
)
```

4. Ngôn ngữ SQL

Các ràng buộc toàn vẹn

❖ PRIMARY KEY

Ràng buộc PRIMARY KEY xác định sự tồn tại và duy nhất của bản ghi trong bảng.

Ví dụ:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT pk_PersonID PRIMARY KEY (P_Id,LastName)
)
```

4. Ngôn ngữ SQL

Các ràng buộc toàn vẹn

❖ FOREIGN KEY/ REFERENCES

Ràng buộc FOREIGN KEY xác định khóa ngoài cho bảng.

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	2
4	24562	1

```
CREATE TABLE Orders
(
  O_Id int NOT NULL,
  OrderNo int NOT NULL,
  P_Id int,
  PRIMARY KEY (O_Id),
  CONSTRAINT fk_PerOrders FOREIGN KEY (P_Id)
  REFERENCES Persons(P_Id)
)
```

4. Ngôn ngữ SQL

4.3 Lệnh sửa bảng

- ❖ Được dùng để
 - ❖ Thay đổi cấu trúc bảng
 - ❖ Thay đổi RBTV

- ❖ **Thêm cột**

ALTER TABLE <Tên_bảng> **ADD COLUMN**
<Tên_cột> <Kiểu_dữ_liệu> [<RBTV>]

Ví dụ:

ALTER TABLE SupplyProduct ADD COLUMN price real NOT NULL;

SupplyProduct

sid	pid	quantity
S1	P1	500
S1	P2	400
S1	P3	100
S2	P2	200
S3	P4	100
S2	P3	155



SupplyProduct

sid	pid	quantity	price
S1	P1	500	1000
S1	P2	400	2300
S1	P3	100	1000
S2	P2	200	1100
S3	P4	100	1000
S2	P3	155	1350

4. Ngôn ngữ SQL

4.3 Lệnh sửa bảng

❖ **Xóa cột** **ALTER TABLE** <Tên_bảng> **DROP COLUMN** <Tên_cột>

Ví dụ:

ALTER TABLE SupplyProduct DROP COLUMN price ;

SupplyProduct

sid	pid	quantity	price
S1	P1	500	1000
S1	P2	400	2300
S1	P3	100	1000
S2	P2	200	1100
S3	P4	100	1000
S2	P3	155	1350

SupplyProduct

sid	pid	quantity
S1	P1	500
S1	P2	400
S1	P3	100
S2	P2	200
S3	P4	100
S2	P3	155

4. Ngôn ngữ SQL

4.3 Lệnh sửa bảng

❖ Sửa kiểu dữ liệu trên một cột

```
ALTER TABLE <Tên_bảng> ALTER COLUMN  
    <Tên_cột> <Kiểu_dữ_liệu_mới>
```

- ❖ Ví dụ: Thay đổi kiểu dữ liệu của trường DateOfBirth của bảng Persons từ kiểu DateTime thành Year.

```
ALTER TABLE Persons  
ALTER COLUMN DateOfBirth year
```

4. Ngôn ngữ SQL

4.3 Lệnh sửa bảng

❖ Thêm RBTV

```
ALTER TABLE <Tên_bảng> ADD  
    CONSTRAINT <Ten_RBTV> <RBTV>,  
    CONSTRAINT <Ten_RBTV> <RBTV>,  
    ...
```

❖ Ví dụ:

```
ALTER TABLE Persons  
ADD CONSTRAINT chk_Person CHECK (P_Id>0 AND City='Sandnes')
```

```
ALTER TABLE Persons  
ADD CONSTRAINT pk_PersonID PRIMARY KEY (P_Id, LastName)
```

4. Ngôn ngữ SQL

4.3 *Lệnh sửa bảng*

❖ Xóa RBTV

```
ALTER TABLE <Tên_bảng> DROP <Tên_RBTV>
```

❖ Ví dụ:

MySQL:

```
ALTER TABLE Persons  
DROP PRIMARY KEY
```

SQL Server / Oracle / MS Access:

```
ALTER TABLE Persons  
DROP CONSTRAINT pk_PersonID
```

4. Ngôn ngữ SQL

4.3 Lệnh xóa bảng

- ❖ Được dùng để xóa cấu trúc bảng
 - ❖ Tất cả dữ liệu của bảng cũng bị xóa
- ❖ Cú pháp
- ❖ Xóa dữ liệu của bảng, vẫn giữ cấu trúc bảng, cú pháp:

DROP TABLE <Tên_bảng>

TRUNCATE TABLE <Tên_bảng>

4. Ngôn ngữ SQL

4.4 Lệnh tạo miền giá trị

- ❖ Tạo ra một kiểu dữ liệu mới kế thừa những kiểu dữ liệu có sẵn
- ❖ Cú pháp

```
CREATE DOMAIN <Tên_kdl_mới> AS <Kiểu_dữ_liệu>
```

- ❖ Ví dụ

```
CREATE DOMAIN Kieu_Ten AS VARCHAR(30)
```

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

- ❖ Gồm 3 mệnh đề

SELECT <danh sách các cột>

FROM <danh sách các bảng>

WHERE <điều kiện>

- ❖ <danh sách các cột>
 - ❖ Tên các cột cần được hiển thị trong kết quả truy vấn
- ❖ <danh sách các bảng>
 - ❖ Tên các bảng liên quan đến câu truy vấn
- ❖ <điều kiện>
 - ❖ Biểu thức boolean xác định dòng nào sẽ được rút trích
 - ❖ Nối các biểu thức: AND, OR, và NOT
 - ❖ Phép toán: < , > , ≤ , ≥ , ≠ , =, LIKE và BETWEEN

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

Dữ liệu mẫu:

Supplier

sid	sname	size	city
S1	Dustin	100	London
S2	Rusty	70	Paris
S3	Lubber	120	London
S4	Pitts	80	Los Angeles

Product

pid	pname	colour
P1	Screw	red
P2	Screw	green
P3	Nut	red
P4	Bolt	blue

SupplyProduct

sid	pid	quantity
S1	P1	500
S1	P2	400
S1	P3	100
S2	P2	200
S3	P4	100
S2	P3	155

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

a. Tìm kiếm không điều kiện trên một bảng

```
SELECT <danh sách các cột>  
FROM <Tên bảng>
```

```
SELECT *  
FROM <Tên bảng>
```

❖ Ví dụ:

Đưa ra tên của các mặt hàng

```
Select pname  
From Product;
```

Kết quả truy vấn:

pname
Screw
Screw
Nut
Bolt

Đưa ra toàn bộ thông tin về các hãng
cung ứng

```
Select *  
From Supplier;
```

Supplier			
sid	sname	size	city
S1	Dustin	100	London
S2	Rusty	70	Paris
S3	Lubber	120	London
S4	Pitts	80	Los Angeles

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

Từ khóa **DISTINCT**

Loại bỏ các giá trị trùng lặp.

Ví dụ:

Đưa ra tên khác nhau của các mặt hàng.

Select Distinct pname

From Product;

Kết quả sau thực hiện câu lệnh:

Product

pid	pname	colour
P1	Screw	red
P2	Screw	green
P3	Nut	red
P4	Bolt	blue

pname
Screw
Nut
Bolt

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

b. Tìm kiếm có điều kiện trên một bảng

SELECT <danh sách các cột>

FROM <Tên bảng>

WHERE <điều kiện>

Ví dụ:

Đưa ra tên của các hãng cung ứng có trụ sở tại London:

Select *sname*

From *Supplier*

Where *city* = "London";

Đưa ra mã số và tên của các hãng cung ứng nằm ở London và có số nhân viên lớn hơn 75:

Select *sid, sname*

From *Supplier*

Where *city* = "London" **AND** *size* > 75;

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Mệnh đề Order By

- ❖ Dùng để hiển thị kết quả câu truy vấn theo một thứ tự nào đó
- ❖ Cú pháp

SELECT <danh sách các cột>

FROM <danh sách các bảng>

WHERE <điều kiện>

❖ **ORDER BY** <danh sách các cột>

❖ ASC: tăng (mặc định)

❖ DESC: giảm

```
SELECT MA_NVIEN, SODA
FROM PHANCONG
ORDER BY MA_NVIEN DESC,
SODA
```

MA_NVIEN	SODA
999887777	10
999887777	30
987987987	10
987987987	30
987654321	10
987654321	20
987654321	30

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

c. Tìm kiếm có điều kiện sử dụng toán tử phạm vi IN và BETWEEN

IN và BETWEEN là mệnh đề sử dụng trong biểu thức điều kiện sau Where để kiểm tra điều kiện thành viên tập hợp hoặc xác định khoảng giá trị.

Ví dụ:

Đưa ra thông tin của các hãng cung ứng có số nhân viên trong khoảng từ 100 đến 150

Select *

From Supplier

Where size BETWEEN 100 AND 150;



sid	sname	size	city
S1	Dustin	100	London
S3	Lubber	120	London

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

c. Tìm kiếm có điều kiện sử dụng toán tử phạm vi IN và BETWEEN

Đưa ra mã số của hãng cung ứng mặt hàng P1 hoặc P2

Cách 1:

Select sid

From SupplyProduct

Where pid = "P1" OR pid = "P2";

Cách 2:

Select sid

From SupplyProduct

Where pid IN ("P1", "P2");

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

c. Tìm kiếm có điều kiện xử lý xâu ký tự (toán tử LIKE)

Toán tử LIKE được sử dụng trong trường hợp cần xử lý xâu ký tự đúng hoặc gần đúng dựa trên phép đối sánh mẫu.

Thường đi kèm với các ký tự đại diện (WildCards)

SQL chuẩn %: thay thế một xâu con (zero or more chars)

_ : thay thế 1 ký tự bất kỳ (exactly one char)

[<charlist>]: thuộc bất kỳ ký tự nào trong dãy kí tự

[^|!<charlist>]: không thuộc bất kỳ ký tự nào trong dãy kí tự liệt kê.

Access: %: thay thế 1 kí tự

* : thay thế cho 1 xâu

Nếu muốn đối sánh dấu % và _ như ký tự bình thường, thêm \ vào trước dấu đó.

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

c. Tìm kiếm có điều kiện xử lý xâu ký tự (toán tử LIKE)

Ví dụ: Bảng "Persons"

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

Đưa ra những người có họ bắt đầu bằng chữ B, S hoặc P.

```
SELECT * FROM Persons  
WHERE LastName LIKE '[bsp]%'
```

Kết quả:

P_Id	LastName	FirstName	Address	City
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

c. Tìm kiếm có điều kiện xử lý chuỗi ký tự (toán tử LIKE)

Ví dụ

Đưa ra thông tin của hãng sản xuất có trụ sở đặt tại thành phố bắt đầu bằng chữ Lo

```
Select *  
From Supplier  
Where city LIKE "Lo%";
```

Supplier

sid	sname	size	city
S1	Dustin	100	London
S2	Rusty	70	Paris
S3	Lubber	120	London
S4	Pitts	80	Los Angeles

Kết quả truy vấn:

sid	sname	size	city
S1	Dustin	100	London
S3	Lubber	120	London
S4	Pitts	80	Los Angeles

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

d. Truy vấn trên nhiều bảng:

SELECT <danh sách các cột> FROM <danh sách các bảng> WHERE <điều kiện>

Điều kiện sau mệnh đề WHERE luôn luôn phải có điều kiện để kết nối các bảng.

Ví dụ:

Đưa ra tên của hãng có cung ứng mặt hàng P1

Select *sname*

From *Supplier , SupplyProduct*

Where *Supplier.sid = SupplyProduct.sid AND pid = "P1";*

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

e. Phép toán đổi tên (SQL Alias) với từ khóa AS

Để đơn giản hóa câu truy vấn, SQL cho phép đổi tên các bảng và các cột trong một câu truy vấn sử dụng cấu trúc <tên cũ> AS <tên mới>.

Rất hữu ích đối với các bảng hoặc các cột có tên dài, phức tạp; hoặc để làm rõ nghĩa một số tên viết tắt...

SQL Alias Syntax for Tables

```
SELECT column_name(s)
FROM table_name
AS alias_name
```

SQL Alias Syntax for Columns

```
SELECT column_name AS alias_name
FROM table_name
```

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

e. Phép toán đổi tên (SQL Alias) với từ khóa AS

Ví dụ:

```
SELECT po.OrderID, p.LastName, p.FirstName  
FROM Persons AS p,  
Product_Orders AS po  
WHERE p.LastName='Hansen' AND p.FirstName='Ola'
```

Nếu không dùng phép đổi tên:

```
SELECT Product_Orders.OrderID, Persons.LastName, Persons.FirstName  
FROM Persons,  
Product_Orders  
WHERE Persons.LastName='Hansen' AND Persons.FirstName='Ola'
```

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

f. Phép toán tập hợp

- ❖ SQL có cài đặt các phép toán

- ❖ Hợp (UNION)

- ❖ Giao (INTERSECT)

- ❖ Trừ (EXCEPT)

- ❖ Kết quả trả về là tập hợp

- ❖ Loại bỏ các bộ trùng nhau

- ❖ Để giữ lại các bộ trùng nhau phải sử dụng từ khóa

- ❖ UNION ALL

- ❖ INTERSECT ALL

- ❖ EXCEPT ALL

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Phép toán tập hợp

Ví dụ UNION

"Employees_Norway":

E_ID	E_Name
01	Hansen, Ola
02	Svendson, Tove
03	Svendson, Stephen
04	Pettersen, Kari

"Employees_USA":

E_ID	E_Name
01	Turner, Sally
02	Kent, Clark
03	Svendson, Stephen
04	Scott, Stephen

```
SELECT E_Name FROM Employees_Norway  
UNION  
SELECT E_Name FROM Employees_USA
```



E_Name
Hansen, Ola
Svendson, Tove
Svendson, Stephen
Pettersen, Kari
Turner, Sally
Kent, Clark
Scott, Stephen

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Phép toán tập hợp

Ví dụ: UNION ALL

"Employees_Norway":

E_ID	E_Name
01	Hansen, Ola
02	Svendson, Tove
03	Svendson, Stephen
04	Pettersen, Kari

"Employees_USA":

E_ID	E_Name
01	Turner, Sally
02	Kent, Clark
03	Svendson, Stephen
04	Scott, Stephen

```
SELECT E_Name FROM Employees_Norway  
UNION ALL  
SELECT E_Name FROM Employees_USA
```



E_Name
Hansen, Ola
Svendson, Tove
Svendson, Stephen
Pettersen, Kari
Turner, Sally
Kent, Clark
Svendson, Stephen
Scott, Stephen

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

g. Truy vấn với các hàm thư viện

Các hàm thư viện trong SQL:

- ❖ COUNT()

- ❖ COUNT(*) đếm số dòng

- ❖ COUNT(<tên thuộc tính>) đếm số giá trị khác NULL của thuộc tính

- ❖ COUNT(DISTINCT <tên thuộc tính>) đếm số giá trị khác nhau và khác NULL của thuộc tính

- ❖ MIN(<tên cột>) trả về giá trị cực tiểu chứa trong cột

- ❖ MAX(<tên cột>) trả về giá trị cực đại chứa trong cột

- ❖ SUM(<tên cột>) tổng giá trị trong cột

- ❖ AVG(<tên cột>) giá trị trung bình của cột

Các hàm kết hợp được đặt ở mệnh đề SELECT

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

g. Truy vấn với các hàm thư viện

Ví dụ:

1. Có tổng cộng bao nhiêu nhân viên làm cho các hãng ở London

Select Sum(size)

From Supplier

Where city = "London";



Sum(size)
220

2. Xem có bao nhiêu hãng có trụ sở ở London

Select Count(city)

From Supplier

Where city = "London";



Count(city)
2

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

h. Tìm kiếm có phân nhóm các bản ghi kết quả

- ❖ Phân nhóm các bản ghi kết quả theo giá trị của một hoặc nhiều thuộc tính

- ❖ Cú pháp

SELECT <danh sách các cột>

FROM <danh sách các bảng>

WHERE <điều kiện>

GROUP BY <danh sách các cột gom nhóm>

- ❖ Sau khi gom nhóm, mỗi nhóm các bộ sẽ có cùng giá trị tại các thuộc tính gom nhóm

- ❖ Mệnh đề GROUP BY

- ❖ Các thuộc tính trong mệnh đề SELECT (trừ những thuộc tính trong các hàm kết hợp) phải xuất hiện trong mệnh đề GROUP BY

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

h. Tìm kiếm có phân nhóm các bản ghi kết quả: bảng Orders

O_Id	OrderDate	OrderPrice	Customer
1	2008/11/12	1000	Hansen
2	2008/10/23	1600	Nilsen
3	2008/09/02	700	Hansen
4	2008/09/03	300	Hansen
5	2008/08/30	2000	Jensen
6	2008/10/04	100	Nilsen

```
SELECT Customer,SUM(OrderPrice) FROM Orders  
GROUP BY Customer
```

The result-set will look like this:

Customer	SUM(OrderPrice)
Hansen	2000
Nilsen	1700
Jensen	2000

```
SELECT Customer,SUM(OrderPrice) FROM Orders
```

The result-set will look like this:

Customer	SUM(OrderPrice)
Hansen	5700
Nilsen	5700
Hansen	5700
Hansen	5700
Jensen	5700
Nilsen	5700

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Điều kiện trên nhóm HAVING

Sử dụng để giới hạn số nhóm hiển thị của câu truy vấn.

SELECT <danh sách các cột>

FROM <danh sách các bảng>

WHERE <điều kiện>

GROUP BY <danh sách các cột gom nhóm>

HAVING <điều kiện trên nhóm>

❖ Mệnh đề HAVING

- ❖ Sử dụng các hàm kết hợp trong mệnh đề SELECT để kiểm tra một số điều kiện nào đó
- ❖ Chỉ kiểm tra điều kiện trên nhóm, không là điều kiện lọc trên từng bộ
- ❖ Sau khi gom nhóm, điều kiện trên nhóm mới được thực hiện

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Điều kiện trên nhóm HAVING

Ví dụ: Bảng Orders

Tìm và đưa ra tất cả khách hàng có tổng giá trị các hóa đơn <2000

```
SELECT Customer, SUM(OrderPrice) FROM Orders  
GROUP BY Customer  
HAVING SUM(OrderPrice) < 2000
```

The result-set will look like this:

Customer	SUM(OrderPrice)
Nilsen	1700

Tìm khách hàng có tên "Hansen" hoặc "Jensen" có tổng giá trị các hóa đơn >1500

```
SELECT Customer, SUM(OrderPrice) FROM Orders  
WHERE Customer='Hansen' OR Customer='Jensen'  
GROUP BY Customer  
HAVING SUM(OrderPrice) > 1500
```



Customer	SUM(OrderPrice)
Hansen	2000
Jensen	2000

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Truy vấn lồng

- ❖ Các câu lệnh **SELECT** có thể lồng nhau ở nhiều mức
- ❖ Câu truy vấn con thường trả về một tập các giá trị
- ❖ Kết quả của câu truy vấn con sẽ làm điều kiện cho câu truy vấn ngoài, thứ tự thực thi từ trong ra ngoài
- ❖ Cú pháp :

SELECT <danh sách các cột>

FROM <danh sách các bảng>

WHERE < điều kiện so sánh> (**SELECT**....

FROM

WHERE...)

T/vấn ngoài

Truy vấn con

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Truy vấn lồng

- ❖ Các câu truy vấn trong cùng một mệnh đề WHERE được kết hợp bằng phép nối logic
- ❖ Mệnh đề WHERE của câu truy vấn ngoài
 - ❖ <biểu thức> <so sánh tập hợp> <truy vấn con>
 - ❖ So sánh tập hợp thường đi cùng với một số toán tử
 - ❖ IN, NOT IN
 - ❖ ALL
 - ❖ ANY hoặc SOME
 - ❖ Kiểm tra sự tồn tại
 - ❖ EXISTS
 - ❖ NOT EXISTS

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Truy vấn lồng

Ví dụ:

Đưa ra mã số của các mặt hàng được cung ứng bởi các hãng có trụ sở tại London

```
Select distinct pid  
From SupplyProduct  
Where sid IN (Select sid From Supplier where city = "London");
```

Đưa ra tên của các hãng không cung ứng mặt hàng P3

```
Select sname  
From Supplier  
Where sid NOT IN (Select sid  
                    from SupplyProduct Where pid = "P3");
```

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Truy vấn lồng

Ví dụ:

Đưa ra thông tin của các hãng cung ứng ít nhất 1 mặt hàng

Select * From Supplier S

Where Exists (select * from SupplyProduct SP
where SP.sid = S.sid) ;

Đưa ra thông tin của các nhà cung cấp không cung ứng mặt hàng nào

Select * From Supplier S

Where Not Exists (select * from SupplyProduct SP
where SP.sid = S.sid) ;

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

❖ Truy vấn lồng

❖ Lồng phân cấp

- ❖ Mệnh đề WHERE của truy vấn trong không tham chiếu đến thuộc tính của các quan hệ trong mệnh đề FROM ở truy vấn ngoài
- ❖ Khi thực hiện, câu truy vấn trong sẽ được thực hiện trước

❖ Lồng tương quan

- ❖ Mệnh đề WHERE của truy vấn trong tham chiếu ít nhất một thuộc tính của các quan hệ trong mệnh đề FROM ở truy vấn ngoài
- ❖ Khi thực hiện, câu truy vấn trong sẽ được thực hiện nhiều lần, mỗi lần tương ứng với một bộ của truy vấn ngoài

4. Ngôn ngữ SQL

4.5 Truy vấn dữ liệu

- ❖ Cho phép kiểm tra điều kiện và xuất thông tin theo từng trường hợp
- ❖ Cú pháp

CASE <tên cột>

WHEN <giá trị> **THEN** <biểu thức>

WHEN <giá trị> **THEN** <biểu thức>

...

[**ELSE** <biểu thức>]

END

4. Ngôn ngữ SQL

4.6 Cập nhật dữ liệu

❖ Lệnh INSERT

- ❖ Dùng để thêm 1 hay nhiều dòng vào bảng
- ❖ Để thêm dữ liệu
- ❖ Tên quan hệ
- ❖ Danh sách các thuộc tính cần thêm dữ liệu
- ❖ Danh sách các giá trị tương ứng

❖ Cú pháp

INSERT INTO <tên bảng>(<danh sách các thuộc tính>)

VALUES (<danh sách các giá trị>)

4. Ngôn ngữ SQL

4.6 Cập nhật dữ liệu

❖ Lệnh INSERT

- ❖ Thứ tự các giá trị phải trùng với thứ tự các cột
- ❖ Có thể thêm giá trị NULL ở những thuộc tính không là khóa chính và NOT NULL
- ❖ Câu lệnh INSERT sẽ gặp lỗi nếu vi phạm RBTV
 - ❖ Khóa chính
 - ❖ Tham chiếu
 - ❖ NOT NULL - các thuộc tính có ràng buộc NOT NULL bắt buộc phải có giá trị

4. Ngôn ngữ SQL

4.6 Cập nhật dữ liệu

❖ Lệnh INSERT

Ví dụ: Thêm dữ liệu vào bảng Persons

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

```
INSERT INTO Persons  
VALUES (4,'Nilsen', 'Johan', 'Bakken 2', 'Stavanger')
```

The "Persons" table will now look like this:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger

4. Ngôn ngữ SQL

4.6 Cập nhật dữ liệu

❖ Lệnh INSERT

Ví dụ: Thêm dữ liệu vào bảng Persons

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

```
INSERT INTO Persons (P_Id, LastName, FirstName)  
VALUES (5, 'Tjessem', 'Jakob')
```

The "Persons" table will now look like this:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jakob		

4. Ngôn ngữ SQL

4.6 Cập nhật dữ liệu

❖ Lệnh DELETE

- ❖ Dùng để xóa các dòng của bảng (xóa mọi dòng, xóa có điều kiện)

❖ Cú pháp

DELETE FROM <tên bảng>
[**WHERE** <điều kiện>]

- ❖ Có thể dùng DELETE * FROM <table> : xóa mọi dòng
- ❖ Ví dụ:

Delete From SupplyProduct **where** sid = 'S3';

4. Ngôn ngữ SQL

4.6 Cập nhật dữ liệu

Lệnh UPDATE

- ❖ Dùng để thay đổi giá trị của thuộc tính cho các dòng của bảng
- ❖ Cú pháp

UPDATE <tên bảng>

SET <tên thuộc tính>=<giá trị mới>,

<tên thuộc tính>=<giá trị mới>,

...

[**WHERE** <điều kiện sửa đổi>]

4. Ngôn ngữ SQL

4.6 Cập nhật dữ liệu

❖ Lệnh UPDATE

Ví dụ:

Hãng S1 chuyển tới Milan

Update Supplier **Set** city = "Milan"

Where sid = "S1";

sid	sname	size	city
S1	Dustin	100	Milan
S2	Rusty	70	Paris
S3	Lubber	120	London
S4	Pitts	80	Los Angeles

Tất cả các mặt hàng được cung cấp với số lượng nhỏ hơn 100 đều tăng số lượng lên 1.5 lần

Update SupplyProduct **Set** quantity = quantity * 1.5

Where quantity < 100;

sid	pid	quantity
S1	P1	500
S1	P2	400
S1	P3	150
S2	P2	200
S3	P4	150
S2	P3	155

4. Ngôn ngữ SQL

4.7 Khung nhìn

❖ Cú pháp

```
CREATE VIEW <tên khung nhìn> AS  
    <câu truy vấn>
```

```
DROP VIEW <tên khung nhìn>
```

❖ Bảng ảo này có

- ❖ Danh sách thuộc tính trùng với các thuộc tính trong mệnh đề SELECT
- ❖ Số dòng phụ thuộc vào điều kiện ở mệnh đề WHERE
- ❖ Dữ liệu được lấy từ các bảng ở mệnh đề FROM

4. Ngôn ngữ SQL

4.7 Khung nhìn

- ❖ Có thể dùng các câu lệnh INSERT, DELETE và UPDATE cho các khung nhìn đơn giản
 - ❖ Khung nhìn được xây dựng trên 1 bảng và có khóa chính của bảng
- ❖ Không thể cập nhật dữ liệu nếu
 - ❖ Khung nhìn có dùng từ khóa DISTINCT
 - ❖ Khung nhìn có sử dụng các hàm kết hợp
 - ❖ Khung nhìn có mệnh đề SELECT mở rộng
 - ❖ Khung nhìn được xây dựng từ bảng có RB trên cột
 - ❖ Khung nhìn được xây dựng từ nhiều bảng

4. Ngôn ngữ SQL

4.8 Chỉ mục

- ❖ Chỉ mục trên thuộc tính A là một cấu trúc dữ liệu làm cho việc tìm kiếm mẫu tin có chứa A hiệu quả hơn

SELECT *

FROM NHANVIEN

WHERE PHG=5 AND GT='Nu'

Đọc 10.000 bộ

Đọc 200 bộ

Bảng NHANVIEN có 10.000 bộ

Có 200 nhân viên làm việc cho phòng 5

Đọc 70 bộ

4. Ngôn ngữ SQL

4.8 Chỉ mục

❖ Cú pháp

```
CREATE INDEX <tên chỉ mục> ON <tên bảng>(<tên cột>)
```

```
DROP INDEX <tên chỉ mục>
```

❖ Ví dụ

```
CREATE INDEX PHG_IND ON NHANVIEN(PHG)
```

```
CREATE INDEX PHG_GT_IND ON NHANVIEN(PHG, GT)
```

4. Ngôn ngữ SQL

4.8 Chỉ mục

- ❖ Tìm kiếm nhanh trong trường hợp so sánh với hằng số và phép kết nối
 - ❖ Làm chậm đi các thao tác thêm, xóa và sửa
 - ❖ Tốn chi phí
 - ❖ Lưu trữ chỉ mục
 - ❖ Truy xuất đĩa nhiều
- ❖ Chọn lựa cài đặt chỉ mục hợp lý



HẾT CHƯƠNG IV

Thanks for your listening!